Early Detection of Silent Training Failures in Neural Networks

Sohan Bhat
sohanrambhat@gmail.com

November 2026

Abstract

SOHAN WRITES THIS

1 Introduction

SOHAN WRITES THIS

2 Related Work

SOHAN WRITES THIS

3 Method

SOHAN WRITES THIS

$$x < a_{\ell se}(1-m) \quad \text{or} \quad x > b_{\ell se}(1+m) + f \tag{1}$$

4 Experimental Setup

SOHAN WRITES THIS

5 Results

5.1 LHD and the accuracy curve

SOHAN WRITES THIS

5.2 Faults that cost accuracy

SOHAN WRITES THIS

5.3 Dead units and fixed thresholds

SOHAN WRITES THIS

Table 1: SOHAN WRITES THIS

When the alarmed layer shows	LHD names
a signal that is not a finite number	diverged
more dead units than its range allows	dead units
weight change below its range while the gradient is inside or above	frozen layer
weight change and gradient both below their ranges	vanishing gradient
weight norm above its range	runaway weights
any other pattern	unrecognized

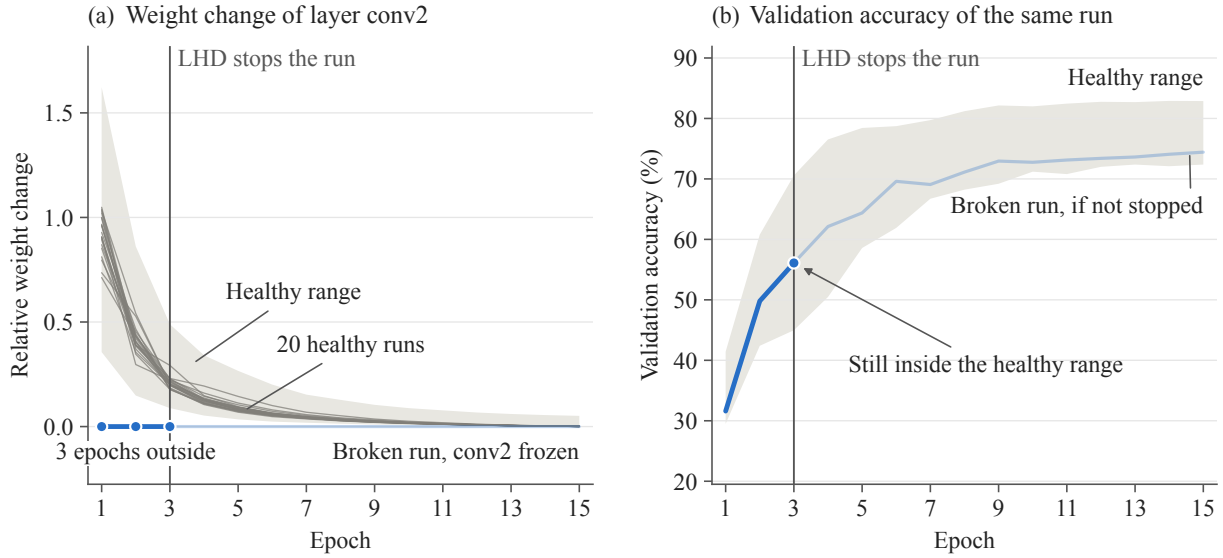


Figure 1: SOHAN WRITES THIS

5.4 False alarms

SOHAN WRITES THIS

6 Limitations

SOHAN WRITES THIS

7 Conclusion

SOHAN WRITES THIS

Table 2: SOHAN WRITES THIS

Network	Dataset	Parameters	Epochs	Calibration	Fresh healthy	Frozen or dead	Other faults
Fully connected	Fashion-MNIST	242,762	20	20	50	130	55
Transformer	Tiny Shakespeare	112,577	15	20	50	170	85
CNN	GTSRB	41,659	15	20	50	170	65
Total				60	150	470	205

Table 3: SOHAN WRITES THIS

	Fully connected		Transformer		CNN	
	LHD	Accuracy	LHD	Accuracy	LHD	Accuracy
Broken runs caught	**84%**	26%	**100%**	48%	**78%**	29%
Median epoch of the alarm	**3**	17	**3**	5	**3**	7
Healthy runs flagged	1 of 50	1 of 50	**0 of 50**	6 of 50	2 of 50	**0 of 50**

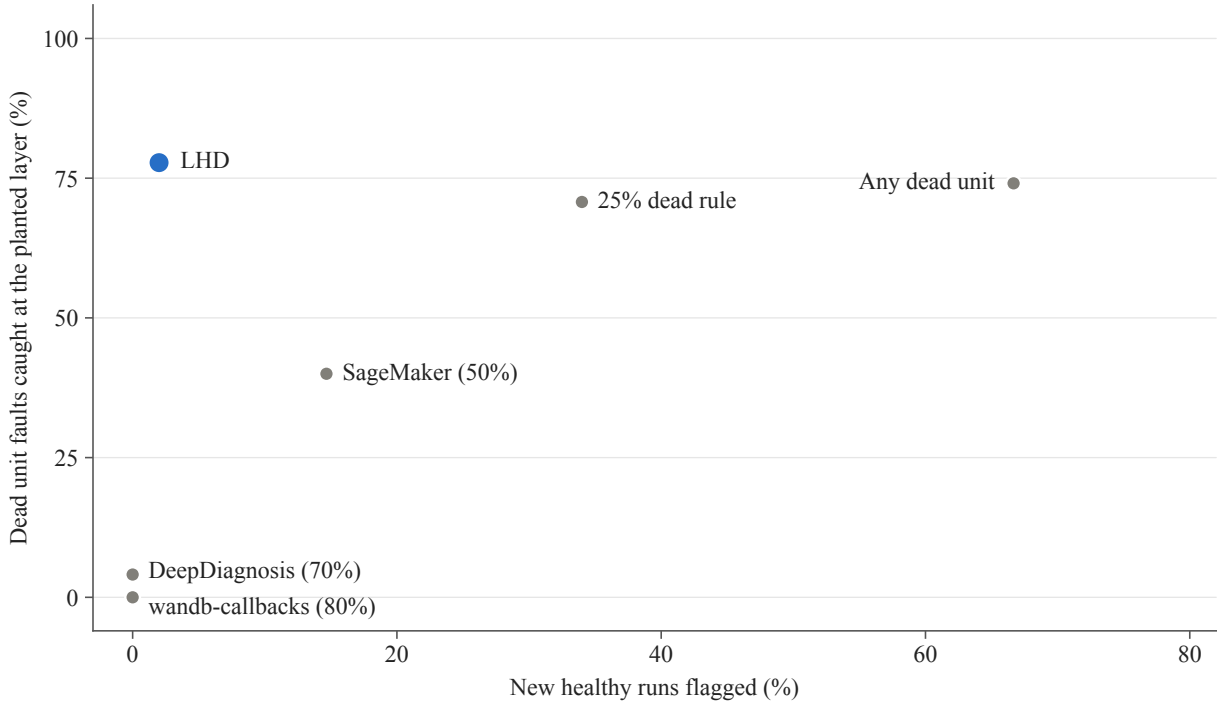


Figure 2: SOHAN WRITES THIS

Use of generative AI

SOHAN WRITES THIS

Code and data

SOHAN WRITES THIS

References

Amazon Web Services (n.d.a). List of Debugger built-in rules. Amazon SageMaker AI Developer Guide. https://docs.aws.amazon.com/sagemaker/latest/dg/debugger-built-in-rules.html

Amazon Web Services (n.d.b). Use Debugger built-in actions for rules. Amazon SageMaker AI Developer Guide. https://docs.aws.amazon.com/sagemaker/latest/dg/debugger-built-in-actions.html

Biewald, L. (2020). Experiment tracking with Weights and Biases. Software available from wandb.com. https://www.wandb.com/

Chwilczyński, S., Trębacz, K., Cyganik, K., Małecki, M., and Brzezinski, D. (2024). ART: Actually Robust Training. In *Machine Learning and Knowledge Discovery in Databases. Research Track and Demo Track (ECML PKDD 2024)*, Lecture Notes in Computer Science, vol. 14948, pp. 374–378. Springer, Cham. https://doi.org/10.1007/978-3-031-70371-3_23. Preprint: arXiv:2408.16285.

CS231n (n.d.). Neural networks part 3: Learning and evaluation. Course notes for CS231n, Stanford University. https://cs231n.github.io/neural-networks-3/

Groeger, F. (n.d.). wandb-callbacks: Additional callbacks for Weights & Biases. GitHub repository. https://github.com/FabianGroeger96/wandb-callbacks

Karpathy, A. (2019). A recipe for training neural networks. Blog post, April 25, 2019. https://karpathy.github.io/2019/04/25/recipe/

Kim, J., Humbatova, N., Jahangirova, G., Tonella, P., and Yoo, S. (2023). Repairing DNN architecture: Are we there yet? In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pp. 234–245. IEEE. https://doi.org/10.1109/ICST57152.2023.00030. Preprint: arXiv:2301.11568.

Schoop, E., Huang, F., and Hartmann, B. (2021). UMLAUT: Debugging deep learning programs using program structure and model behavior. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1–16. ACM. https://doi.org/10.1145/3411764.3445538

Stallkamp, J., Schlipsing, M., Salmen, J., and Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32:323–332. https://doi.org/10.1016/j.neunet.2012.02.016

Wardat, M., Le, W., and Rajan, H. (2021). DeepLocalize: Fault localization for deep neural networks. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 251–262. IEEE. https://doi.org/10.1109/ICSE43902.2021.00034

Wardat, M., Cruz, B. D., Le, W., and Rajan, H. (2022). DeepDiagnosis: Automatically diagnosing faults and recommending actionable fixes in deep learning programs. In *Proceedings of the 44th International Conference on Software Engineering (ICSE 2022)*, pp. 561–572. ACM, New York. https://doi.org/10.1145/3510003.3510071. Preprint: arXiv:2112.04036.

Zhang, X., Zhai, J., Ma, S., and Shen, C. (2021). AUTOTRAINER: An automatic DNN training problem detection and repair system. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 359–371. IEEE. https://doi.org/10.1109/ICSE43902.2021.00043

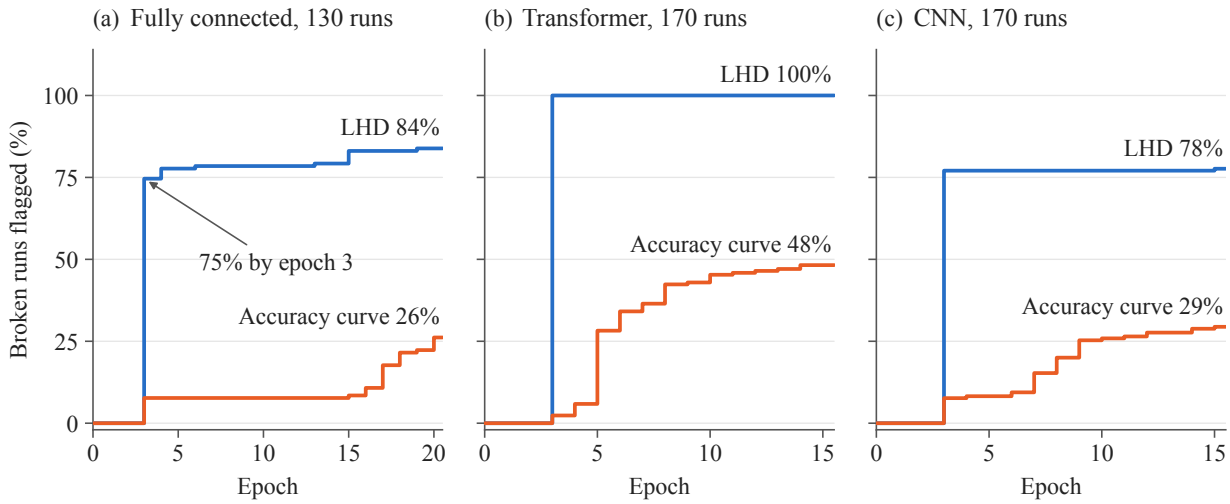


Figure A1: SOHAN WRITES THIS

Table A1: SOHAN WRITES THIS

	Caught at the planted layer		
Detector	Frozen layers	Dead units	Healthy runs flagged
LHD	97%	**78%**	3 of 150
Zero update check	**100%**	0%	0 of 150
25% dead rule	0%	71%	51 of 150
SageMaker Debugger (50%)	0%	40%	22 of 150
DeepDiagnosis (70%)	0%	4%	0 of 150
wandb-callbacks (80%)	0%	0%	0 of 150
Any dead unit (wandb-callbacks verbose)	2%	74%	100 of 150

A Networks and runs in detail

SOHAN WRITES THIS

B Detection over time

SOHAN WRITES THIS

C Every detector

SOHAN WRITES THIS

D Each layer and how many units must die

SOHAN WRITES THIS

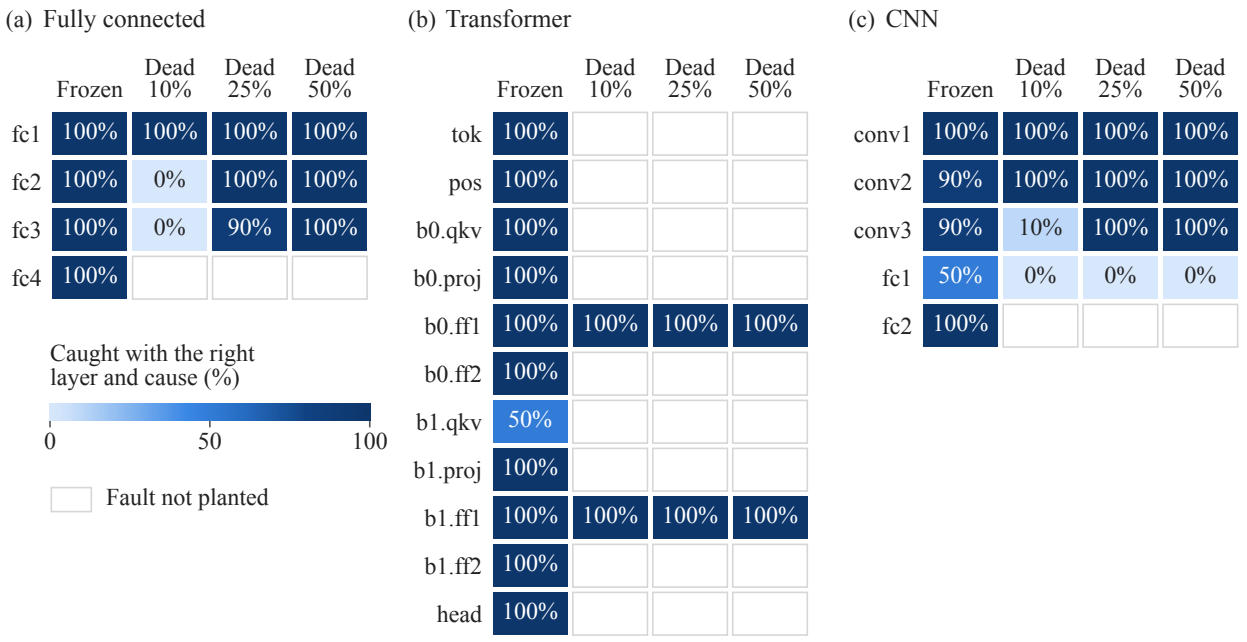


Figure A2: SOHAN WRITES THIS

E What each signal adds

SOHAN WRITES THIS

F Margin and calibration size

SOHAN WRITES THIS

G Cost

SOHAN WRITES THIS

H Faults beyond frozen layers and dead units

SOHAN WRITES THIS

I Statistics for each seed and condition

SOHAN WRITES THIS

Table A2: SOHAN WRITES THIS

Signals used	Broken runs caught		Right layer and cause	
	Rate	Change	Rate	Change
All four	87%		85%	
Without weight change	70%	−18	45%	−40
Without weight norm	87%	same	85%	same
Without grad norm	87%	same	86%	+1
Without dead unit fraction	43%	−45	41%	−44
Only weight change	43%	−45	43%	−42
Only weight norm	10%	−77	0%	−85
Only grad norm	25%	−63	0%	−85
Only dead unit fraction	52%	−36	45%	−40

Table A3: SOHAN WRITES THIS

Fault	Runs	LHD	Accuracy curve	Zero update check	50% dead rule
Frozen layer at epoch 6	100	100%	20%	100%	0%
50% dead units at epoch 6	45	**89%**	16%	0%	82%
Gradient cut	30	100%	100%	100%	0%
Learning rate $\times 10$ at epoch 6	15	**100%**	33%	0%	0%
Learning rate $\times 100$ at epoch 6	15	100%	100%	67%	60%
Healthy runs flagged	150	3	7	**0**	22